Escuela Da Vinci

Carrera Analista de Sistemas · Seminario Final

GoTrain

Aplicación social de fitness con Coach IA

Requisitos No Funcionales

Alumno: Santiago Ghiano

Versión 2.0 · 11/11/2026

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	24/09/2026	Santiago Ghiano	Versión inicial (Parcial 1, sobre prototipos).
2.0	11/11/2026	Santiago Ghiano	Actualización según la implementación final (Parcial 2).

Índice

Control de versiones 2
Índice 2
1. Introducción 3
2. Requisitos no funcionales 3
3. Restricciones técnicas 3

1. Introducción

Los requisitos no funcionales definen las cualidades del sistema GoTrain: seguridad, usabilidad, rendimiento, portabilidad, mantenibilidad y disponibilidad. Para cada uno se indica cómo se verifica.

2. Requisitos no funcionales

ID	Categoría	Descripción	Verificación
RNF-01	Seguridad	Las contraseñas se almacenan con hash (scrypt mediante Werkzeug); nunca en texto plano.	Revisión de models.py / tabla usuarios
RNF-02	Seguridad	Todos los endpoints, salvo registro, login y health, exigen un token JWT firmado (HS256) con vencimiento de 7 días.	CP-06, CP-07
RNF-03	Seguridad	Un usuario solo puede leer y modificar sus propios entrenamientos, ejercicios, rutinas y mensajes.	CP-32
RNF-04	Seguridad	La clave de la Claude API y la clave de firma se configuran por variables de entorno (.env) y no se suben al repositorio.	.gitignore / .env.example
RNF-05	Usabilidad	Interfaz en español rioplatense, tema oscuro con color de acento #E94560, coherente con los prototipos.	Manuales de usuario
RNF-06	Usabilidad	Validación en tiempo real y mensajes de error claros en formularios.	CP-33, CP-38
RNF-07	Usabilidad / Portabilidad	La web es responsive (escritorio y celular, quiebre a 900 px).	styles.css
RNF-08	Portabilidad	App mobile para Android 7.0+ distribuida como APK; el mismo código React Native permite generar la versión iOS.	APK
RNF-09	Interoperabilidad	Comunicación cliente-servidor por API REST con JSON sobre HTTP, compartida por web y mobile.	Diseño de arquitectura
RNF-10	Disponibilidad	Si la Claude API no responde o falta la clave, el Coach sigue funcionando en modo demo.	RF-23 / CP-25
RNF-11	Rendimiento	Las operaciones habituales de la API responden en menos de 500 ms en la red local (las 32 pruebas de API corren en unos 19 s, incluyendo la creación de la base).	Ejecución de pruebas
RNF-12	Rendimiento	El Coach con IA responde en pocos segundos; mientras tanto la interfaz muestra "El coach está escribiendo…".	Coach
RNF-13	Mantenibilidad	Separación en capas (presentación, API, lógica, datos) y ORM SQLAlchemy, que permite pasar de SQLite a PostgreSQL cambiando solo DATABASE_URL.	Arquitectura
RNF-14	Mantenibilidad	Código versionado en Git y pruebas automatizadas reproducibles.	Repositorio / tests
RNF-15	Escalabilidad	API sin estado (stateless) gracias a JWT: permite correr varias instancias detrás de un balanceador.	Arquitectura
RNF-16	Privacidad	Al Coach IA solo se envían nombre, nivel, ciudad e historial de entrenamientos: no se envían email ni contraseña.	coach.py

3. Restricciones técnicas

- Backend: Python 3.13, Flask 3, Flask-SQLAlchemy, PyJWT.
- Base de datos: PostgreSQL 17 (producción) / SQLite (desarrollo y pruebas).

- Web: React 18 + Vite 6. Mobile: React Native con Expo SDK 58.
- IA: Claude API (modelo claude-opus-5-5) con salida estructurada en JSON.
- Navegadores soportados: Chrome, Edge y Firefox actuales.